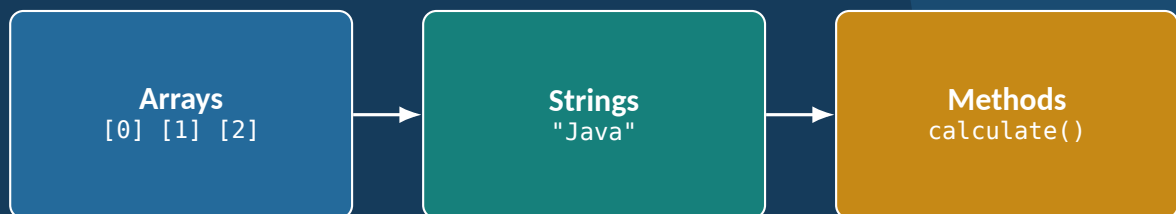


Understanding Arrays, Strings, and Methods in Java

A visual and practical journey from foundational concepts to intermediate problem solving



Soban Hussain

Teaching Assistant | The University of Larkano

Read the idea → visualize the working → trace the code → implement the solution
Practise independently and solve confidently.

```
knowledge.learn();  
knowledge.implement();  
knowledge.solve();
```

Contents

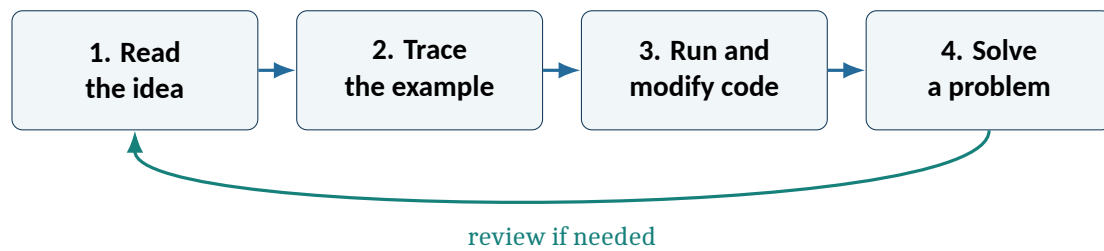
1	How to Use This Guide	4
2	The Big Picture	4
I	Understanding Arrays	5
3	Why Do We Need Arrays?	5
4	Visual Model of an Array	5
5	Declaring, Creating, and Initializing Arrays	6
5.1	Declaration	6
5.2	Creation	6
5.3	Declaration and creation together	6
5.4	Initializer syntax	6
5.5	Default values	6
6	Accessing and Updating Elements	7
7	The length Property	7
8	Traversing Arrays with Loops	7
8.1	Traditional for loop	7
8.2	Enhanced for loop	8
9	Taking Array Input	8
10	Core Array Problem-Solving Patterns	8
10.1	Total and average	8
10.2	Maximum and minimum	9
10.3	Counting values that satisfy a condition	9
10.4	Linear search	9
10.5	Reverse traversal	10
11	Tracing an Array Algorithm	10
12	Two-Dimensional Arrays	10
13	Complete Array Program: Marks Analyzer	11

II Understanding Strings	12
14 What Is a String?	12
15 Useful String Methods	13
16 Reading Strings with Scanner	13
16.1 The nextInt and nextLine issue	13
17 Comparing Strings Correctly	13
18 Traversing a String	14
19 String Problem: Count Vowels	14
20 String Problem: Reverse Text	15
21 String Problem: Palindrome Test	15
22 Strings and Arrays Together	15
22.1 Array of strings	15
22.2 Split a sentence into words	15
III Understanding Methods	16
23 Why Do We Need Methods?	16
24 Anatomy of a Method	16
25 Four Common Method Forms	17
25.1 No parameters and no return value	17
25.2 Parameters but no return value	17
25.3 No parameters but returns a value	17
25.4 Parameters and a return value	17
26 Parameters and Arguments	17
27 Return Values	18
28 Passing Arrays to Methods	18
29 Passing Strings to Methods	18
30 Scope of Variables	19

31 Method Overloading	19
32 Decomposing a Problem into Methods	20
IV Integrated Implementation	20
33 Complete Program: Student Result System	20
34 A General Problem-Solving Framework	22
35 Easy Practice Problems	22
36 Intermediate Practice Problems	23
37 Guided Challenge: Top Student	23
38 Selected Solutions	24
38.1 Count even and odd values	24
38.2 Count occurrences of a target	24
38.3 Find the longest word	24
38.4 Complete the top-student comparison	25
39 Common Errors Checklist	25
40 Final Self-Assessment	25
41 Final Advice	26

1 How to Use This Guide

This document is designed for independent study as well as classroom and lab use. Do not try to memorize every program. Instead, use the following learning cycle:



Learning objectives

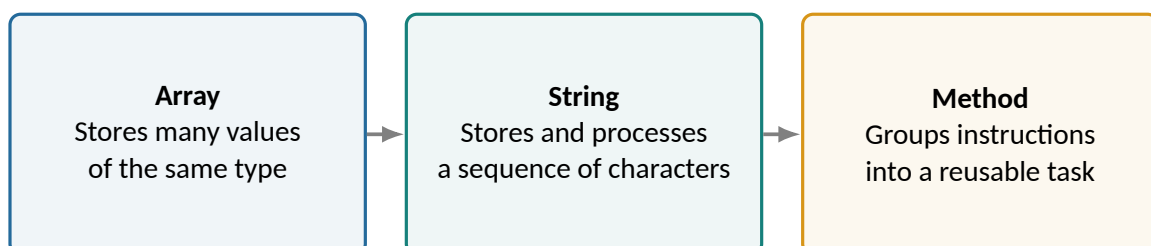
After completing the guide, you should be able to:

- create, initialize, access, update, and traverse one-dimensional and two-dimensional arrays;
- solve array problems such as total, average, maximum, minimum, counting, searching, and reversing;
- create and manipulate Java `String` values correctly;
- compare strings, inspect characters, count vowels, reverse text, and test palindromes;
- define and call methods with parameters and return values;
- pass arrays and strings to methods;
- divide a larger problem into small reusable methods; and
- combine arrays, strings, and methods in complete programs.

Programming becomes easier when a large problem is divided into small, understandable steps.

2 The Big Picture

Arrays, strings, and methods solve three different but connected problems.



Example: a student-result program may use an array to store marks, a string to store the student's name, and methods to calculate and display the result.

```
1 String studentName = "Ali";
2 int[] marks = {78, 85, 69, 90, 81};
3
4 int total = calculateTotal(marks);
5 double average = calculateAverage(marks);
6 displayResult(studentName, total, average);
```

Part I

Understanding Arrays

3 Why Do We Need Arrays?

Suppose a program stores marks in five subjects without an array:

```
1 int mark1 = 80;
2 int mark2 = 75;
3 int mark3 = 90;
4 int mark4 = 65;
5 int mark5 = 88;
```

This is manageable for five values, but it becomes difficult for 50 or 500 values. An array provides one name for a group of related values.

```
1 int[] marks = {80, 75, 90, 65, 88};
```

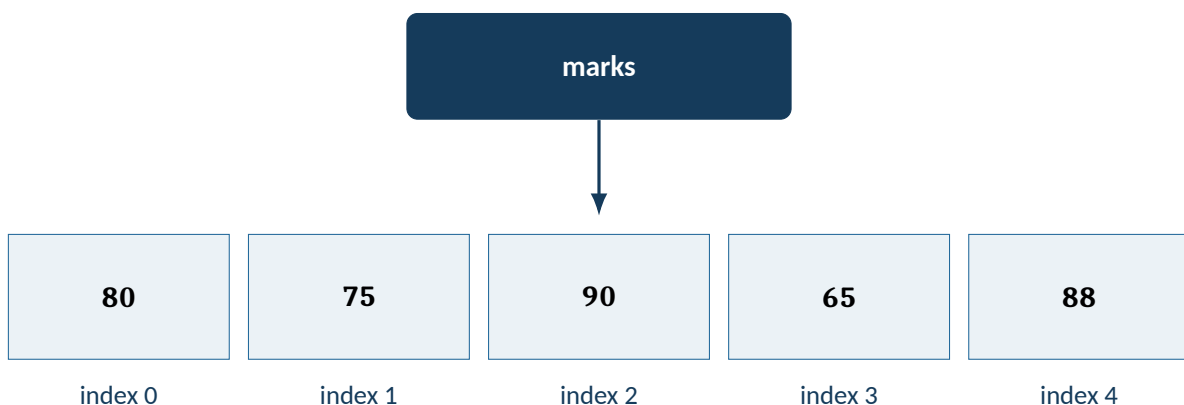
Definition

An **array** is a fixed-size collection of values of the same data type. Each value is called an **element**, and each element is accessed through an **index**.

4 Visual Model of an Array

Consider the following array:

```
1 int[] marks = {80, 75, 90, 65, 88};
```



Java starts array indexing at 0. The last valid index is `array.length - 1`.

Checkpoint. For an array of length 8, the first index is 0 and the last index is 7.

5 Declaring, Creating, and Initializing Arrays

5.1 Declaration

```
1 int[] marks;  
2 String[] names;  
3 double[] prices;
```

At this point, the variables can refer to arrays, but the array objects have not yet been created.

5.2 Creation

```
1 marks = new int[5];  
2 names = new String[3];  
3 prices = new double[10];
```

5.3 Declaration and creation together

```
1 int[] marks = new int[5];
```

5.4 Initializer syntax

Use an initializer when the values are already known:

```
1 int[] marks = {80, 75, 90, 65, 88};  
2 String[] cities = {"Larkana", "Sukkur", "Karachi"};
```

5.5 Default values

When an array is created with `new`, Java assigns default values.

Array type	Example	Default element value
<code>int[]</code>	<code>new int[4]</code>	0
<code>double[]</code>	<code>new double[4]</code>	0.0
<code>boolean[]</code>	<code>new boolean[4]</code>	false
<code>char[]</code>	<code>new char[4]</code>	null character
<code>String[]</code>	<code>new String[4]</code>	null

6 Accessing and Updating Elements

```
1 int[] marks = {80, 75, 90};
2
3 System.out.println(marks[0]); // 80
4 System.out.println(marks[2]); // 90
5
6 marks[1] = 85;
7 System.out.println(marks[1]); // 85
```

The expression `marks[1]` means “the value stored at index 1 of the marks array.”

Common mistake. If an array contains three elements, `marks[3]` is invalid. The valid indexes are 0, 1, and 2. An invalid index causes `ArrayIndexOutOfBoundsException`.

Try it yourself. Create an integer array containing 10, 20, 30, and 40. Print the first and last values. Then change 20 to 25.

7 The length Property

Every array has a `length` property.

```
1 int[] marks = {80, 75, 90, 65};
2
3 System.out.println(marks.length); // 4
4 System.out.println(marks[marks.length - 1]); // 65
```

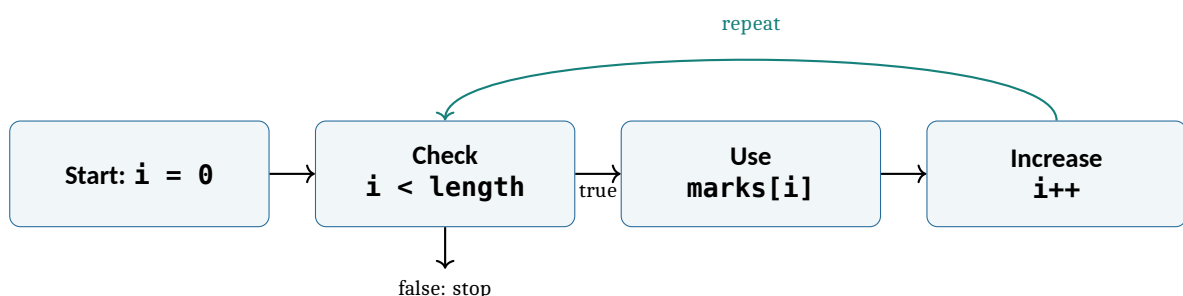
Use `length` instead of writing a fixed number in loops. The program will then continue to work if the array size changes.

8 Traversing Arrays with Loops

8.1 Traditional for loop

Use this form when the index is needed.

```
1 int[] marks = {80, 75, 90, 65};
2
3 for (int i = 0; i < marks.length; i++) {
4     System.out.println("Index " + i + " = " + marks[i]);
5 }
```



8.2 Enhanced for loop

Use this form when only the values are needed.

```
1 for (int mark : marks) {
2     System.out.println(mark);
3 }
```

Read it as: “For each mark in marks, print mark.”

Loop	Best use	Limitation
Traditional for	Index is required; elements may be updated	Slightly more code
Enhanced for	Every value is read in order	Index is not directly available

9 Taking Array Input

```
1 import java.util.Scanner;
2
3 public class ArrayInput {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6         int[] marks = new int[5];
7
8         for (int i = 0; i < marks.length; i++) {
9             System.out.print("Enter mark " + (i + 1) + ": ");
10            marks[i] = input.nextInt();
11        }
12
13        System.out.println("Entered marks:");
14        for (int mark : marks) {
15            System.out.println(mark);
16        }
17    }
18 }
```

Checkpoint. Why is `i + 1` used in the prompt? The array index starts at 0, but users normally count items from 1.

10 Core Array Problem-Solving Patterns

These patterns appear repeatedly in easy and intermediate programs.

10.1 Total and average

```
1 int total = 0;
2
3 for (int mark : marks) {
4     total += mark;
5 }
6
```

```
7 double average = (double) total / marks.length;
```

The variable `total` is an **accumulator**: it stores a running result.

10.2 Maximum and minimum

```
1 int maximum = marks[0];
2 int minimum = marks[0];
3
4 for (int mark : marks) {
5     if (mark > maximum) {
6         maximum = mark;
7     }
8
9     if (mark < minimum) {
10        minimum = mark;
11    }
12 }
```

Do not automatically start the maximum at zero. If all values are negative, zero would produce the wrong result. Starting with the first element is safer.

10.3 Counting values that satisfy a condition

```
1 int passCount = 0;
2
3 for (int mark : marks) {
4     if (mark >= 50) {
5         passCount++;
6     }
7 }
```

10.4 Linear search

```
1 int target = 90;
2 int foundIndex = -1;
3
4 for (int i = 0; i < marks.length; i++) {
5     if (marks[i] == target) {
6         foundIndex = i;
7         break;
8     }
9 }
10
11 if (foundIndex == -1) {
12     System.out.println("Value not found");
13 } else {
14     System.out.println("Found at index " + foundIndex);
15 }
```

The value `-1` is used because it cannot be a valid array index.

10.5 Reverse traversal

```
1 for (int i = marks.length - 1; i >= 0; i--) {  
2     System.out.println(marks[i]);  
3 }
```

11 Tracing an Array Algorithm

Trace this array: {4, 7, 2, 9}.

```
1 int total = 0;  
2 int maximum = numbers[0];  
3  
4 for (int number : numbers) {  
5     total += number;  
6     if (number > maximum) {  
7         maximum = number;  
8     }  
9 }
```

Iteration	number	total after addition	maximum
1	4	4	4
2	7	11	7
3	2	13	7
4	9	22	9

Tracing helps you understand how variable values change after every iteration.

12 Two-Dimensional Arrays

A two-dimensional array stores values in rows and columns. It is useful for matrices, seating plans, monthly records, or marks for multiple students and subjects.

```
1 int[][] marks = {  
2     {80, 75, 90},  
3     {65, 88, 72},  
4     {91, 84, 79}  
5 };
```

	col 0	col 1	col 2
row 0	80	75	90
row 1	65	88	72
row 2	91	84	79

Accessing one element:

```
1 System.out.println(marks[1][2]); // 72
```

Traversing every element requires nested loops:

```
1 for (int row = 0; row < marks.length; row++) {
2     for (int col = 0; col < marks[row].length; col++) {
3         System.out.print(marks[row][col] + " ");
4     }
5     System.out.println();
6 }
```

13 Complete Array Program: Marks Analyzer

```
1 import java.util.Scanner;
2
3 public class MarksAnalyzer {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6         int[] marks = new int[5];
7
8         for (int i = 0; i < marks.length; i++) {
9             System.out.print("Enter mark " + (i + 1) + ": ");
10            marks[i] = input.nextInt();
11        }
12
13        int total = 0;
14        int maximum = marks[0];
15        int minimum = marks[0];
16        int passes = 0;
17
18        for (int mark : marks) {
19            total += mark;
20
21            if (mark > maximum) maximum = mark;
22            if (mark < minimum) minimum = mark;
23            if (mark >= 50) passes++;
24        }
25
26        double average = (double) total / marks.length;
27    }
```

```
28     System.out.println("Total: " + total);
29     System.out.println("Average: " + average);
30     System.out.println("Maximum: " + maximum);
31     System.out.println("Minimum: " + minimum);
32     System.out.println("Passed subjects: " + passes);
33 }
34 }
```

How to improve this program

1. Reject marks below 0 or above 100.
2. Display the index of the highest mark.
3. Count failed subjects.
4. Move each calculation into a separate method after studying methods.

Part II

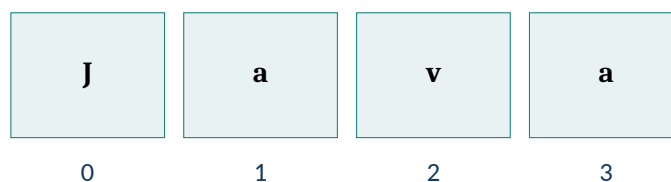
Understanding Strings

14 What Is a String?

A String stores a sequence of characters such as a name, sentence, email address, or password.

```
1 String name = "Soban";
2 String university = "The University of Larkano";
3 String message = "Welcome to Java programming";
```

String text = "Java"



Like an array, a string's characters have indexes beginning at 0.

Important difference

An array has a public field named `length`. A string uses a method named `length()`.

```
1 int[] values = {10, 20, 30};
2 String word = "Java";
3
4 System.out.println(values.length); // array
5 System.out.println(word.length()); // string
```

15 Useful String Methods

Method	Example	Purpose
<code>length()</code>	<code>name.length()</code>	Number of characters
<code>charAt(i)</code>	<code>name.charAt(0)</code>	Character at index <code>i</code>
<code>toUpperCase()</code>	<code>name.toUpperCase()</code>	Uppercase copy
<code>toLowerCase()</code>	<code>name.toLowerCase()</code>	Lowercase copy
<code>equals(text)</code>	<code>name.equals("Ali")</code>	Case-sensitive content comparison
<code>equalsIgnoreCase(text)</code>	<code>name.equalsIgnoreCase("ali")</code>	Comparison ignoring case
<code>contains(text)</code>	<code>sentence.contains("Java")</code>	Tests whether text occurs
<code>startsWith(text)</code>	<code>name.startsWith("S")</code>	Tests the beginning
<code>endsWith(text)</code>	<code>file.endsWith(".java")</code>	Tests the ending
<code>substring(a,b)</code>	<code>word.substring(0,3)</code>	Characters from <code>a</code> to before <code>b</code>
<code>indexOf(text)</code>	<code>word.indexOf("a")</code>	First matching index or <code>-1</code>
<code>trim()</code>	<code>input.trim()</code>	Removes surrounding spaces
<code>replace(a,b)</code>	<code>text.replace('a','x')</code>	Replaces characters
<code>split(pattern)</code>	<code>line.split(" ")</code>	Breaks text into an array

16 Reading Strings with Scanner

```

1 Scanner input = new Scanner(System.in);
2
3 System.out.print("Enter your full name: ");
4 String fullName = input.nextLine();

```

`next()` reads one word. `nextLine()` reads the complete line including spaces.

16.1 The `nextInt` and `nextLine` issue

```

1 System.out.print("Enter age: ");
2 int age = input.nextInt();
3 input.nextLine(); // consumes the remaining newline
4
5 System.out.print("Enter full name: ");
6 String name = input.nextLine();

```

Without the extra `nextLine()`, the name may appear to be skipped.

17 Comparing Strings Correctly

Common mistake. Do not use `==` to compare string content. It compares references, not the actual sequence of characters.

Incorrect:

```

1 if (name == "Ali") {
2     System.out.println("Welcome");
3 }

```

Correct:

```

1 if (name.equals("Ali")) {
2     System.out.println("Welcome");

```

```
3 }
```

Case-insensitive comparison:

```
1 if (answer.equalsIgnoreCase("yes")) {  
2     System.out.println("Continuing...");  
3 }
```

18 Traversing a String

Use `charAt(index)` to inspect one character at a time.

```
1 String word = "Java";  
2  
3 for (int i = 0; i < word.length(); i++) {  
4     System.out.println(i + " -> " + word.charAt(i));  
5 }
```

Output:

```
1 0 -> J  
2 1 -> a  
3 2 -> v  
4 3 -> a
```

19 String Problem: Count Vowels

```
1 String text = "Programming is interesting";  
2 text = text.toLowerCase();  
3  
4 int vowelCount = 0;  
5  
6 for (int i = 0; i < text.length(); i++) {  
7     char character = text.charAt(i);  
8  
9     if (character == 'a' || character == 'e' ||  
10        character == 'i' || character == 'o' ||  
11        character == 'u') {  
12        vowelCount++;  
13    }  
14 }  
15  
16 System.out.println("Vowels: " + vowelCount);
```

Reasoning steps

1. Convert the text to lowercase so each vowel needs only one comparison.
2. Visit every character.
3. Test whether the current character is a vowel.
4. Increase the counter when the condition is true.

20 String Problem: Reverse Text

```
1 String text = "Java";
2 String reversed = "";
3
4 for (int i = text.length() - 1; i >= 0; i--) {
5     reversed += text.charAt(i);
6 }
7
8 System.out.println(reversed); // avaJ
```

For short beginner programs, string concatenation is acceptable. For repeated changes in larger programs, `StringBuilder` is more efficient.

```
1 String reversed = new StringBuilder(text).reverse().toString();
```

21 String Problem: Palindrome Test

A palindrome reads the same forward and backward, such as `level`, `madam`, or `radar`.

```
1 String word = "Level";
2 String normalized = word.toLowerCase();
3 String reversed = "";
4
5 for (int i = normalized.length() - 1; i >= 0; i--) {
6     reversed += normalized.charAt(i);
7 }
8
9 if (normalized.equals(reversed)) {
10     System.out.println("Palindrome");
11 } else {
12     System.out.println("Not a palindrome");
13 }
```

22 Strings and Arrays Together

22.1 Array of strings

```
1 String[] students = {"Ali", "Sara", "Ahmed", "Hina"};
2
3 for (String student : students) {
4     System.out.println(student.toUpperCase());
5 }
```

22.2 Split a sentence into words

```
1 String sentence = "Java makes problem solving interesting";
2 String[] words = sentence.split(" ");
3
4 System.out.println("Word count: " + words.length);
5
```

```
6 for (String word : words) {  
7     System.out.println(word);  
8 }
```

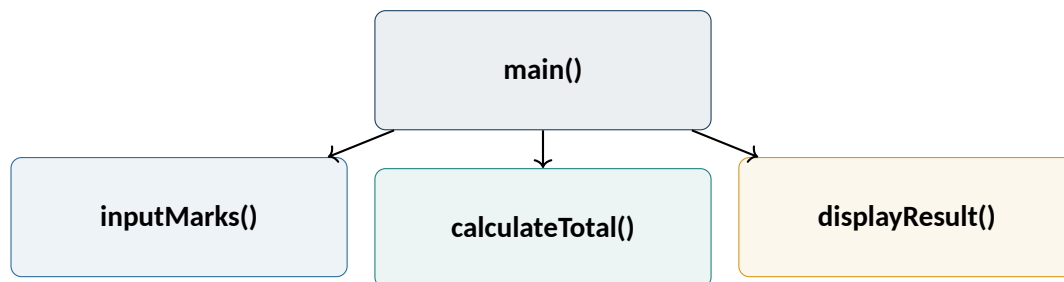
Try it yourself. Read a sentence and count how many words have more than four characters.

Part III

Understanding Methods

23 Why Do We Need Methods?

If every instruction is written inside `main()`, a program becomes difficult to read and test. Methods divide the work into meaningful tasks.



Benefits include:

- less repeated code;
- clearer program structure;
- easier testing and debugging;
- reusable logic; and
- easier collaboration on larger programs.

24 Anatomy of a Method

```
1 static int add(int first, int second) {  
2     int result = first + second;  
3     return result;  
4 }
```



Calling the method:

```

1 int answer = add(10, 20);
2 System.out.println(answer); // 30

```

25 Four Common Method Forms

25.1 No parameters and no return value

```

1 static void showTitle() {
2     System.out.println("Student Result System");
3 }

```

25.2 Parameters but no return value

```

1 static void greet(String name) {
2     System.out.println("Welcome " + name);
3 }

```

25.3 No parameters but returns a value

```

1 static int getPassingMark() {
2     return 50;
3 }

```

25.4 Parameters and a return value

```

1 static int square(int number) {
2     return number * number;
3 }

```

Parameters?	Return value?	Typical purpose
No	No	Display a fixed heading
Yes	No	Display or modify using supplied data
No	Yes	Produce a fixed or internally calculated value
Yes	Yes	Transform input into a result

26 Parameters and Arguments

```

1 static void greet(String name) { // name is a parameter
2     System.out.println("Hello " + name);
3 }
4
5 greet("Ali"); // "Ali" is an argument
6 greet("Sara"); // "Sara" is an argument

```

A **parameter** is a variable in the method definition. An **argument** is the actual value supplied during a method call.

27 Return Values

```
1 static double calculateAverage(int total, int count) {  
2     return (double) total / count;  
3 }  
4  
5 double average = calculateAverage(420, 5);
```

The declared return type and returned value must be compatible.

Common mistake. A `void` method cannot return a result value. A non-void method must return an appropriate value on every possible execution path.

28 Passing Arrays to Methods

```
1 static int calculateTotal(int[] values) {  
2     int total = 0;  
3  
4     for (int value : values) {  
5         total += value;  
6     }  
7  
8     return total;  
9 }
```

Calling the method:

```
1 int[] marks = {80, 75, 90, 65, 88};  
2 int total = calculateTotal(marks);
```

The method works with arrays of different lengths because it uses `values.length` or an enhanced loop rather than a fixed number.

29 Passing Strings to Methods

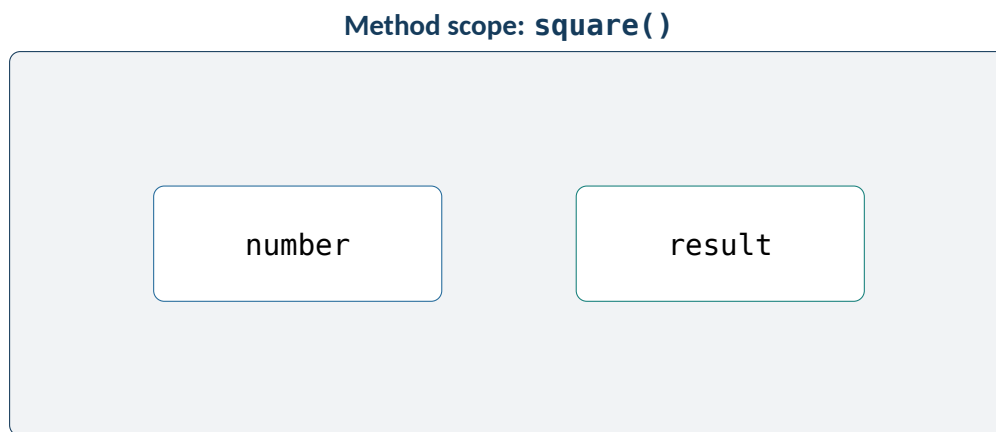
```
1 static int countVowels(String text) {  
2     int count = 0;  
3     text = text.toLowerCase();  
4  
5     for (int i = 0; i < text.length(); i++) {  
6         char ch = text.charAt(i);  
7  
8         if (ch == 'a' || ch == 'e' || ch == 'i' ||  
9             ch == 'o' || ch == 'u') {  
10            count++;  
11        }  
12    }  
13  
14    return count;
```

```
15 }  
  
1  int vowels = countVowels("The University of Larkano");  
2  System.out.println(vowels);
```

30 Scope of Variables

```
1  static int square(int number) {  
2      int result = number * number;  
3      return result;  
4  }
```

The variables `number` and `result` are local to `square`. They cannot be accessed directly from `main()`.



31 Method Overloading

Method overloading uses the same method name with different parameter lists.

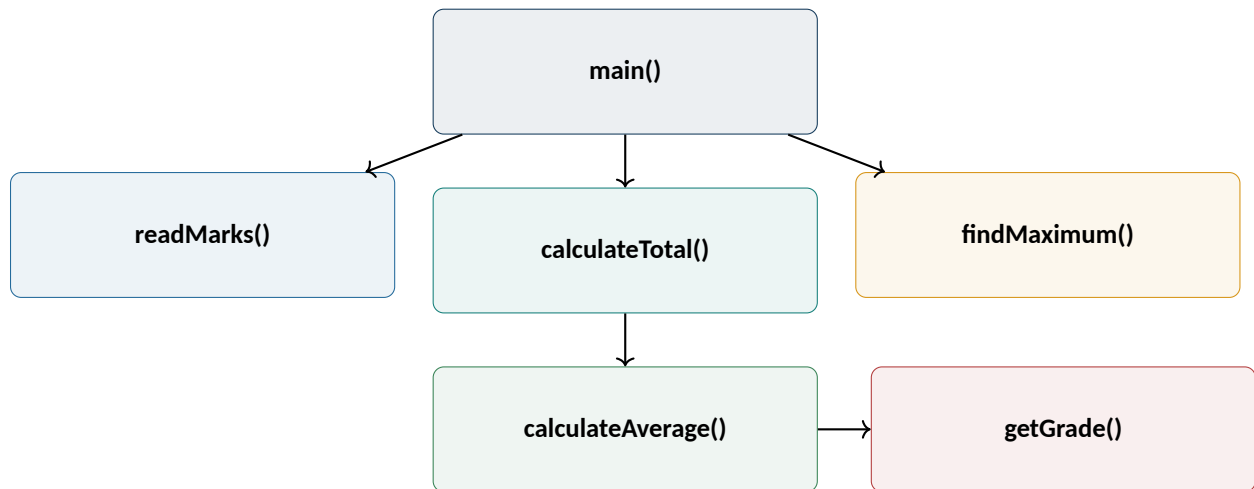
```
1  static int add(int first, int second) {  
2      return first + second;  
3  }  
4  
5  static double add(double first, double second) {  
6      return first + second;  
7  }  
8  
9  static int add(int first, int second, int third) {  
10     return first + second + third;  
11 }
```

Java selects the correct method using the number and types of arguments.

Common mistake. Changing only the return type does not create a valid overload. The parameter list must be different.

32 Decomposing a Problem into Methods

Problem: read five marks and display total, average, maximum, and grade.



This design is easier to test because every method has one main responsibility.

Part IV

Integrated Implementation

33 Complete Program: Student Result System

This program combines a string, an array, loops, conditions, and methods.

```
1 import java.util.Scanner;
2
3 public class StudentResultSystem {
4     static Scanner input = new Scanner(System.in);
5
6     public static void main(String[] args) {
7         System.out.print("Enter student name: ");
8         String name = input.nextLine();
9
10        int[] marks = readMarks(5);
11
12        int total = calculateTotal(marks);
13        double average = calculateAverage(marks);
14        int maximum = findMaximum(marks);
15        int minimum = findMinimum(marks);
16        String grade = getGrade(average);
17
18        displayResult(name, marks, total, average,
19                      maximum, minimum, grade);
20    }
21
22    static int[] readMarks(int count) {
23        int[] marks = new int[count];
24
25        for (int i = 0; i < marks.length; i++) {
```

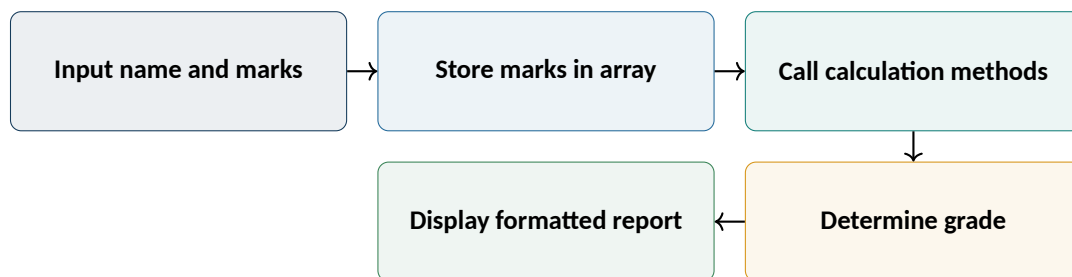
```
26         do {
27             System.out.print("Enter mark " + (i + 1) +
28                             " (0-100): ");
29             marks[i] = input.nextInt();
30         } while (marks[i] < 0 || marks[i] > 100);
31     }
32
33     return marks;
34 }
35
36 static int calculateTotal(int[] marks) {
37     int total = 0;
38
39     for (int mark : marks) {
40         total += mark;
41     }
42
43     return total;
44 }
45
46 static double calculateAverage(int[] marks) {
47     return (double) calculateTotal(marks) / marks.length;
48 }
49
50 static int findMaximum(int[] marks) {
51     int maximum = marks[0];
52
53     for (int mark : marks) {
54         if (mark > maximum) maximum = mark;
55     }
56
57     return maximum;
58 }
59
60 static int findMinimum(int[] marks) {
61     int minimum = marks[0];
62
63     for (int mark : marks) {
64         if (mark < minimum) minimum = mark;
65     }
66
67     return minimum;
68 }
69
70 static String getGrade(double average) {
71     if (average >= 80) return "A";
72     if (average >= 70) return "B";
73     if (average >= 60) return "C";
74     if (average >= 50) return "D";
75     return "F";
76 }
77
78 static void displayResult(String name, int[] marks,
79                           int total, double average,
80                           int maximum, int minimum,
81                           String grade) {
82     System.out.println("\n--- Student Result ---");
83     System.out.println("Name: " + name);
84
85     System.out.print("Marks: ");
```

```

86     for (int mark : marks) {
87         System.out.print(mark + " ");
88     }
89
90     System.out.println("\nTotal: " + total);
91     System.out.printf("Average: %.2f\n", average);
92     System.out.println("Maximum: " + maximum);
93     System.out.println("Minimum: " + minimum);
94     System.out.println("Grade: " + grade);
95 }
96 }

```

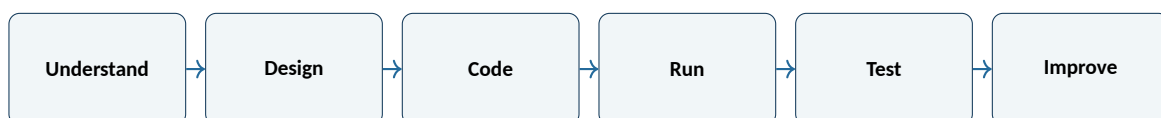
Program structure



34 A General Problem-Solving Framework

Before writing Java code, answer these questions:

1. **What is the input?** Numbers, text, or both?
2. **What is the required output?**
3. **Do I need many values of one type?** Use an array.
4. **Do I need to inspect or transform text?** Use string methods.
5. **Is any task repeated or logically separate?** Create a method.
6. **Do I need a loop?** Decide what changes each iteration.
7. **What conditions must be checked?**
8. **What invalid input or boundary cases can occur?**



35 Easy Practice Problems

Solve these in order. First write the algorithm in plain language, then write Java code.

1. Store five integers in an array and print all values.
2. Read five numbers and display their total and average.

3. Find the largest and smallest number in an array.
4. Count how many values are even and how many are odd.
5. Search for a user-entered number and display its index.
6. Print an array in reverse order.
7. Read a name and display its length, first character, and uppercase form.
8. Count the vowels in a word.
9. Write a method named `isEven` that returns `true` for an even integer.
10. Write a method that receives an integer array and returns its total.

36 Intermediate Practice Problems

1. Read ten marks, reject invalid marks, and display pass and fail counts.
2. Find the second-largest value in an integer array.
3. Create a new array containing the original array in reverse order.
4. Count how many times a target number occurs in an array.
5. Read a sentence and display the longest word.
6. Count vowels, consonants, digits, spaces, and other characters in a string.
7. Test whether a sentence is a palindrome after ignoring spaces and letter case.
8. Write methods for array total, average, maximum, minimum, and search.
9. Store student names in one array and marks in another; display the top student.
10. Use a 2D array to store marks for three students in four subjects, then display each student's average.

37 Guided Challenge: Top Student

Problem: Store five student names and their percentages. Display the name and percentage of the top student.

Step 1: Identify data

- `String[] names`
- `double[] percentages`

Step 2: Identify the algorithm

1. Assume index 0 is the top index.
2. Compare every later percentage with the percentage at the top index.
3. When a larger value is found, update the top index.
4. Use the same index to access both arrays.

Step 3: Complete the missing code

```
1 String[] names = {"Ali", "Sara", "Ahmed", "Hina", "Zain"};
2 double[] percentages = {72.5, 88.0, 79.5, 91.0, 84.5};
3
4 int topIndex = 0;
5
6 for (int i = 1; i < percentages.length; i++) {
7     // Write the comparison and update here.
8 }
9
10 System.out.println("Top student: " + names[topIndex]);
11 System.out.println("Percentage: " + percentages[topIndex]);
```

38 Selected Solutions

Try the exercises before reading this section.

38.1 Count even and odd values

```
1 static void displayEvenOddCounts(int[] values) {
2     int evens = 0;
3     int odds = 0;
4
5     for (int value : values) {
6         if (value % 2 == 0) {
7             evens++;
8         } else {
9             odds++;
10        }
11    }
12
13    System.out.println("Even values: " + evens);
14    System.out.println("Odd values: " + odds);
15 }
```

38.2 Count occurrences of a target

```
1 static int countOccurrences(int[] values, int target) {
2     int count = 0;
3
4     for (int value : values) {
5         if (value == target) {
6             count++;
7         }
8     }
9
10    return count;
11 }
```

38.3 Find the longest word

```

1 static String findLongestWord(String sentence) {
2     String[] words = sentence.trim().split("\\s+");
3     String longest = words[0];
4
5     for (String word : words) {
6         if (word.length() > longest.length()) {
7             longest = word;
8         }
9     }
10
11     return longest;
12 }

```

38.4 Complete the top-student comparison

```

1 for (int i = 1; i < percentages.length; i++) {
2     if (percentages[i] > percentages[topIndex]) {
3         topIndex = i;
4     }
5 }

```

39 Common Errors Checklist

Problem	Incorrect idea	Correct habit
Array boundary	<code>i <= array.length</code>	Use <code>i < array.length</code>
First element	Assume index starts at 1	Start at index 0
String comparison	<code>text == "Java"</code>	Use <code>text.equals("Java")</code>
String length	<code>text.length</code>	Use <code>text.length()</code>
Array length	<code>array.length()</code>	Use <code>array.length</code>
Integer average	<code>total / count</code>	Cast: <code>(double) total / count</code>
Method call	Define but never call method	Call the method where needed
Return type	Return text from an <code>int</code> method	Match the returned value and type
Input line skipped	Call <code>nextLine()</code> after <code>nextInt()</code>	Consume the pending newline
Empty array	Access <code>array[0]</code> without checking	Ensure the array contains elements

40 Final Self-Assessment

Mark each statement when you can perform it without copying a complete solution.

- | | |
|--|---|
| ☐ I can declare and initialize arrays. | ☐ I can find maximum and minimum values. |
| ☐ I can explain indexes and length. | ☐ I can search and count occurrences. |
| ☐ I can traverse arrays with both loop forms. | ☐ I can use a two-dimensional array. |
| ☐ I can calculate totals and averages. | ☐ I can read complete string input. |

- | | |
|---|--|
| ☐ I can compare strings correctly. | ☐ I understand parameters and arguments. |
| ☐ I can traverse string characters. | ☐ I can return a value from a method. |
| ☐ I can count vowels and reverse text. | ☐ I can pass arrays and strings to methods. |
| ☐ I can define and call methods. | ☐ I can divide one problem into methods. |

41 Final Advice

Do not measure progress by how much code you memorize. Measure progress by how clearly you can explain the problem, design the steps, trace the variables, and correct your own mistakes.

For every new program:

1. write the required input and output;
2. solve one small example manually;
3. choose suitable arrays, strings, loops, conditions, and methods;
4. write and test one section at a time;
5. test normal, boundary, and invalid cases; and
6. improve names and structure after the program works.

Understand → Design → Code → Test → Improve

Soban Hussain — Teaching Assistant, The University of Larkano